

Milestone 6 - Submission Checklist

MLOps Course - Module 7

Pre-Submission Verification

Use this checklist to verify your Milestone 6 submission is complete before submitting.

Deliverables Checklist

Part 1: RAG Pipeline Implementation (5 pts)

- ☐ **RAG pipeline notebook/script** (rag_pipeline.ipynb or rag_pipeline.py)
 - ☐ Document ingestion with chunking strategy implemented
 - ☐ Embedding generation using open-source model
 - ☐ Vector index creation (FAISS, Chroma, or equivalent)
 - ☐ Retrieval query execution working
 - ☐ Grounded answer generation functional with a real open-weight 7B-14B instruct LLM
- ☐ **Evaluation report** (rag_evaluation_report.md or PDF, ~2 pages)
 - ☐ Retrieval accuracy on 10 handcrafted queries
 - ☐ Precision@k and/or recall@k metrics included
 - ☐ Qualitative grounding analysis documented
 - ☐ Hallucination cases identified (if any)
 - ☐ Retrieval failures separated from generation/grounding failures
 - ☐ Latency measurements for retrieval, generation, and end-to-end
- ☐ **Pipeline diagram** (rag_pipeline_diagram.md or image file)
 - ☐ Data flow from documents to final output shown
 - ☐ Key components labeled (chunker, embedder, vector store, retriever, generator)
 - ☐ Decision points and data transformations indicated
- ☐ **Environment file** (requirements.txt or lockfile)
 - ☐ All dependencies pinned to specific versions
 - ☐ Open-source vector database dependency included
 - ☐ Model-serving dependency included (Ollama client, vLLM, or Transformers)
- ☐ **Model deployment evidence** (README section or separate markdown file)
 - ☐ Exact model name and version documented
 - ☐ Size class documented (7B, 8B, 14B, etc.)
 - ☐ Serving stack documented
 - ☐ Hardware/runtime environment documented

-
- ☐ Typical generation latency documented

Part 2: Multi-Tool Agent with Retrieval Integration (5 pts)

- ☐ **Agent controller implementation** (agent_controller.py or notebook)
 - ☐ Intelligent tool selection logic implemented
 - ☐ Decision-making logic exposed and observable
 - ☐ At least two tools integrated
 - ☐ Real open-weight 7B-14B instruct LLM used in final evaluated workflow
- ☐ **Retriever tool** integrated from Part 1 or standalone
 - ☐ Callable as agent tool
 - ☐ Returns structured retrieval results
- ☐ **Additional tool** (summarizer, classifier, extractor, reasoning module, etc.)
 - ☐ Functional implementation
 - ☐ Integrated with agent controller
- ☐ **10 multi-step evaluation tasks** with traces (agent_traces/ or in notebook)
 - ☐ Decision points documented for each task
 - ☐ Tool selection logic observable
 - ☐ Retrieval results shown
 - ☐ Final outputs with evidence trails
- ☐ **Agent report** (agent_report.md or PDF, 1-2 pages)
 - ☐ Tool selection policy explained
 - ☐ Retrieval integration described
 - ☐ Performance analysis on 10 tasks
 - ☐ Failure analysis included
 - ☐ Model quality/latency tradeoffs discussed
- ☐ **Environment file** (if different from Part 1)

Combined Deliverables

- ☐ **README.md** with:
 - ☐ Setup instructions (step-by-step)
 - ☐ Usage examples for RAG pipeline
 - ☐ Usage examples for agent controller
 - ☐ Architecture overview
 - ☐ Known limitations documented
 - ☐ Exact model startup/serving commands included

Submission Requirements

- ☐ **New repository** named ids568-milestone6-[your_netid] (e.g., ids568-milestone6-jsmith)
- ☐ All deliverables at **repository root level** (not in subdirectories)
- ☐ Final submission tagged: git tag submission && git push --tags

Alternative submission: If you have constraints preventing a new repository, contact the instructor to arrange branch-based submission.

Rubric Evidence Map

Rubric Criterion	Points	Evidence Location	Verified
Part 1: RAG Pipeline			
Correct implementation of retriever + generator pipeline	2.0	rag_pipeline.ipynb - functional code with ingestion, retrieval, generation using a real open-weight 7B-14B model	[]
Clear evaluation of retrieval accuracy + grounding	1.5	rag_evaluation_report.md - metrics table, grounding analysis section	[]
Explanation of chunking/indexing design decisions	1.0	rag_evaluation_report.md or README - design decisions section	[]
Pipeline diagram clarity and reproducibility	0.5	rag_pipeline_diagram.md + requirements.txt + README model setup section	[]
Part 2: Agent Controller			
Working agent that selects tools in multi-step workflow	2.0	agent_controller.py - autonomous multi-tool coordination with a real open-weight 7B-14B model in the loop	[]
Correct integration of retrieval as decision-triggered tool	1.5	agent_controller.py - retrieval tool invocation logic	[]
Quality of reasoning traces + transparency of decisions	1.0	agent_traces/ - observable decision logic in traces	[]
Evaluation task diversity and documentation	0.5	agent_report.md - 10 diverse tasks documented	[]

Common Pitfalls

Warning: Unoptimized chunking Don't use default chunk sizes without testing. Experiment with different chunk sizes (256, 512, 1024 tokens) and overlap percentages.

Warning: Missing latency measurements Track latency at each pipeline stage separately (retrieval vs. generation vs. end-to-end). Don't just report total time.

Warning: Mock or extractive-only generation Mock responses, template-only outputs, and extractive substitutes are fine for debugging, but they do not satisfy the final submission requirement. Final evaluated runs must use a real open-weight 7B-14B instruct model.

Warning: Opaque agent decisions Log every tool selection and reasoning step. If your agent's decisions aren't observable, you'll lose points on transparency.

Warning: Insufficient evaluation queries Create diverse queries that test different aspects: factual retrieval, multi-hop reasoning, edge cases, out-of-scope queries.

Warning: Missing failure analysis Understanding when and why the agent fails is as important as successes. Document failure cases explicitly.

Warning: Proprietary API dependencies Ensure your solution runs without external proprietary APIs. Use a local or self-hosted open-weight 7B-14B instruct model.

Warning: Non-reproducible environment Pin all dependencies to specific versions. Test that someone can clone and run your code with just `pip install -r requirements.txt`.

Automated Sanity Checks

Run these commands before submitting to verify your submission is complete:

File Existence Checks

```
# Part 1: RAG Pipeline files
echo "=== Part 1: RAG Pipeline Files ==="
for file in "rag_pipeline.ipynb" "rag_pipeline.py"; do
    [ -f "$file" ] && echo "✓ $file exists" || echo "○ $file not found (check
    if using alternative name)"
done

# Check for evaluation report
for file in "rag_evaluation_report.md" "rag_evaluation_report.pdf"; do
    [ -f "$file" ] && echo "✓ $file exists" || echo "○ $file not found"
done
```

```

# Check for pipeline diagram
ls -la *diagram* 2>/dev/null && echo "✓ Diagram file found" || echo "x No
  diagram file found"

# Check for requirements
[ -f "requirements.txt" ] && echo "✓ requirements.txt exists" || echo "x
  requirements.txt missing"

# Part 2: Agent Controller files
echo "=== Part 2: Agent Controller Files ==="
for file in "agent_controller.py" "agent_controller.ipynb"; do
  [ -f "$file" ] && echo "✓ $file exists" || echo "o $file not found (check
    if using alternative name)"
done

# Check for agent report
for file in "agent_report.md" "agent_report.pdf"; do
  [ -f "$file" ] && echo "✓ $file exists" || echo "o $file not found"
done

# Check for agent traces
ls -la agent_traces/ 2>/dev/null || echo "o agent_traces/ directory not
  found (may be in notebook)"

# Combined deliverables
echo "=== Combined Deliverables ==="
[ -f "README.md" ] && echo "✓ README.md exists" || echo "x README.md
  missing"

# Check README contents
if [ -f "README.md" ]; then
  grep -q "Setup" README.md && echo "  ✓ Setup section found" || echo "  x
    Setup section missing"
  grep -q "Usage" README.md && echo "  ✓ Usage section found" || echo "  x
    Usage section missing"
  grep -q -i "limitation" README.md && echo "  ✓ Limitations section found"
  || echo "  x Limitations section missing"
  grep -qiE "(ollama|vllm|transformers)" README.md && echo "  ✓ Model
    serving section found" || echo "  x Model serving details missing"
fi

```

Environment Validation

```

# Validate requirements.txt format
echo "=== Environment Validation ==="
if [ -f "requirements.txt" ]; then

```

```

echo "Checking for pinned versions..."
unpinned=$(grep -E "^[a-zA-Z]" requirements.txt | grep -v "==" | head -5)
if [ -z "$unpinned" ]; then
    echo "✓ All dependencies appear to be pinned"
else
    echo "✗ Some dependencies may not be pinned:"
    echo "$unpinned"
fi

# Check for vector database
grep -iE "(faiss|chroma|weaviate|qdrant|milvus)" requirements.txt && \
    echo "✓ Vector database dependency found" || \
    echo "✗ No recognized vector database in requirements.txt"

# Check for model serving dependency
grep -iE "(transformers|vllm|ollama)" requirements.txt && \
    echo "✓ Model serving dependency found" || \
    echo "○ Model serving package not found in requirements.txt (verify if
        using external Ollama install)"
fi

```

RAG Pipeline Validation

```

# Test RAG pipeline imports (Python)
echo "=== RAG Pipeline Import Check ==="
python3 -c "
import sys
try:
    # Check for common RAG dependencies
    deps = ['langchain', 'llama_index', 'chromadb', 'faiss',
            'sentence_transformers']
    found = []
    for dep in deps:
        try:
            __import__(dep.replace('-', '_'))
            found.append(dep)
        except ImportError:
            pass
    if found:
        print(f'✓ Found RAG dependencies: {found}')
    else:
        print('○ No common RAG dependencies found (may use custom
            implementation)')
except Exception as e:
    print(f'Error: {e}')
"

```

Evaluation Query Count Check

```
# Check for 10 evaluation queries in Part 1
echo "=== Evaluation Query Count ==="
if [ -f "rag_evaluation_report.md" ]; then
    query_count=$(grep -ciE "(query|question)" rag_evaluation_report.md)
    echo "Found approximately $query_count query/question mentions in report"
    [ "$query_count" -ge 10 ] && echo "✓ Likely has 10+ queries" || echo "○
    Verify 10 queries are documented"
fi
```

Agent Trace Validation

```
# Check for 10 agent evaluation tasks
echo "=== Agent Trace Count ==="
if [ -d "agent_traces" ]; then
    trace_count=$(ls -l agent_traces/ 2>/dev/null | wc -l)
    echo "Found $trace_count trace files"
    [ "$trace_count" -ge 10 ] && echo "✓ Has 10+ trace files" || echo "x Need
    at least 10 evaluation tasks"
fi

# Or check in notebook/report
if [ -f "agent_report.md" ]; then
    task_count=$(grep -ciE "(task|evaluation|test case)" agent_report.md)
    echo "Found approximately $task_count task mentions in agent report"
fi
```

Self-Assessment Questions

Before submitting, answer these questions honestly:

Reproducibility

- ☐ **Can someone clone my repo and reproduce the RAG pipeline in 5 minutes?**
 - Clear setup instructions in README?
 - All dependencies in requirements.txt?
 - Sample documents provided or instructions to obtain them?
- ☐ **Can someone run my agent controller without additional setup beyond requirements.txt?**

-
- No hardcoded paths?
 - No proprietary API keys required?
 - Real open-weight 7B-14B model startup documented?
 - Clear usage examples provided?

Part 1: RAG Pipeline Quality

- ☐ **Does my chunking strategy have clear justification?**
 - Did I explain why I chose the chunk size and overlap?
 - Did I test alternatives?
- ☐ **Are my 10 evaluation queries diverse?**
 - Do they test different retrieval scenarios?
 - Are there edge cases (no relevant docs, ambiguous queries)?
- ☐ **Is my grounding analysis substantive?**
 - Did I identify specific hallucination cases?
 - Did I analyze when generation stays grounded vs. drifts?
 - Did I separate retrieval errors from model-generation errors?
- ☐ **Do I have latency measurements for all three stages?**
 - Retrieval latency measured?
 - Generation latency measured?
 - End-to-end latency measured?
 - Model name and serving setup recorded next to those measurements?

Part 2: Agent Controller Quality

- ☐ **Can I observe my agent's decision-making at each step?**
 - Are tool selections logged?
 - Is reasoning visible in traces?
 - Is the live model actually in the evaluated workflow?
- ☐ **Do my 10 tasks require multi-step reasoning?**
 - Do tasks require multiple tool invocations?
 - Are tasks diverse (not just variations of the same query)?
- ☐ **Did I document failure cases?**
 - When does the agent choose the wrong tool?
 - When does retrieval fail to help?
- ☐ **Is my tool selection policy clearly explained?**
 - What triggers retrieval vs. other tools?
 - How does the agent decide when it has enough information?

Integration Quality

- ☐ **Do my RAG and agent components share code appropriately?**
 - Is the retriever reusable between Part 1 and Part 2?
 - Is there unnecessary code duplication?
- ☐ **Is my README complete?**
 - Setup instructions tested on clean environment?
 - Architecture overview understandable by someone new to the project?
 - Known limitations honestly documented?

Final Verification

```
# Complete submission check
echo "=== FINAL SUBMISSION VERIFICATION ==="
echo ""

# Count deliverables
total=0
found=0

# Part 1 checks
for item in "RAG pipeline code" "Evaluation report" "Pipeline diagram"
    "requirements.txt"; do
    total=$((total + 1))
done

[ -f "rag_pipeline.ipynb" ] || [ -f "rag_pipeline.py" ] && found=$((found + 1)) && echo "✓ RAG pipeline code" || echo "✗ RAG pipeline code"
[ -f "rag_evaluation_report.md" ] || [ -f "rag_evaluation_report.pdf" ] && found=$((found + 1)) && echo "✓ Evaluation report" || echo "✗ Evaluation report"
ls *diagram* >/dev/null 2>&1 && found=$((found + 1)) && echo "✓ Pipeline diagram" || echo "✗ Pipeline diagram"
[ -f "requirements.txt" ] && found=$((found + 1)) && echo "✓ requirements.txt" || echo "✗ requirements.txt"

# Part 2 checks
for item in "Agent controller" "Agent report" "Agent traces" "README"; do
    total=$((total + 1))
done

[ -f "agent_controller.py" ] || [ -f "agent_controller.ipynb" ] && found=$((found + 1)) && echo "✓ Agent controller" || echo "✗ Agent controller"
[ -f "agent_report.md" ] || [ -f "agent_report.pdf" ] && found=$((found + 1)) && echo "✓ Agent report" || echo "✗ Agent report"
[ -d "agent_traces" ] || grep -q "trace" *.ipynb 2>/dev/null && found=$((found + 1)) && echo "✓ Agent traces" || echo "○ Agent traces (verify in notebook)"
[ -f "README.md" ] && found=$((found + 1)) && echo "✓ README.md" || echo "✗ README.md"

echo ""
echo "=== Score: $found / $total deliverables found ==="
echo ""
if [ "$found" -eq "$total" ]; then
    echo "✓ All deliverables present - ready for submission!"
```

```
else
  echo "x Some deliverables missing - review checklist above"
fi
```

Submission

Create a new GitHub repository named `ids568-milestone6-[your_netid]` (e.g., `ids568-milestone6-jsmith`).

Ensure all deliverables are at the **repository root level**.

Before submitting:

1. Run all automated sanity checks above
2. Verify all 10 RAG queries and 10 agent tasks are documented
3. Verify the exact open-weight model and serving setup are documented
4. Ensure no proprietary API dependencies (local/self-hosted open-weight only)
5. Tag your final submission: `git tag submission && git push --tags`

Submit via Course Submission Site: Your repository URL

After grades are finalized, you may delete this repository to reduce clutter.

Alternative submission: If you have constraints preventing a new repository, contact the instructor to arrange branch-based submission.

Final Confirmation

- ☐ All code runs without errors on a clean environment
- ☐ All 10 RAG evaluation queries are documented with metrics
- ☐ All 10 agent tasks have observable traces
- ☐ Pipeline diagram accurately represents implementation
- ☐ README provides complete setup and usage instructions
- ☐ Final evaluated runs use a real open-weight 7B-14B instruct model
- ☐ No proprietary API dependencies (local/self-hosted open-weight only)
- ☐ All files are committed and pushed to repository